Computer Security Exam

Professors S. Zanero & M. Carminati & A. Barenghi

09/01/2025

Last (family) Name _____
First (given) Name _____
Matricola _____
Codice Persona _____

Professor: [ ] Zanero [ ] Carminati [ ] Barenghi

Have you done any challenges/homework, even partially? [ ] Yes [ ] No

Do you want to withdraw? [ ] Yes [ ] No

Priority Grading [ ] Yes, Motivation:

[ ] No

Instructions

- **Duration**: 2.5 hours.
- The exam is composed of 19 pages. Check that you have all of them.
- As a cross-check, tell us whether you have completed challenges by appropriately putting an "X" mark.
- The exam is "closed book." The students will not be allowed to consult any **course material** and **notes**.
- **No extra devices** (e.g., phones, iPads) are **allowed**. You will be expelled if, at any time, you do not follow this rule.
- Shut down and store any electronic device. They will be subject to inspection if found, and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good; short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

- **DISTANCE MODE ONLY**
  - On the computer, you can open only the MS Forms browser tab and the exam pdf. Any other application or document is prohibited.
  - You will have to share your screen and leave your microphone open, and your webcam must frame both you and your workspace.
  - You are responsible for having all the necessary technological equipment for the correct exam delivery.
  - If you disable the screen sharing or the audio, you will be expelled from the virtual room, and your exam will be voided.
  - We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or a subset at the instructor's decision.
  - Regarding the submission:
    - At the end of the exam, you will only be given 10 minutes to scan your documents and prepare to upload them.
    - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
    - We are not responsible for any technical issues in the submission (e.g., the exam pictures are not readable, and the pdf is corrupted). We will evaluate only the uploaded readable documents.
    - We will evaluate only the uploaded **readable** documents.
    - USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf).

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 - Introduction to Computer Security (4 points)

Modern cities increasingly rely on intelligent traffic management systems to optimize traffic flow, reduce congestion, and improve public safety. However, connecting critical infrastructure to a network also introduces serious security risks.
Consider the following scenario in a smart city:

- The city operates a central traffic control center that manages traffic signals, digital signage, and highway ramp meters. Communication between traffic lights and the control center is handled via FTP for configuration updates and HMAC for authentication.
- Multiple IoT sensors (vehicle-count sensors, weather sensors, CCTV cameras) feed real-time data to the central system over a legacy Wi-Fi network protected by WEP.
- The back-end system that processes all traffic data runs on an AWS Server and stores information in a legacy SQL database.
- The city's web portal for public traffic updates and government staff access supports HTTPS with TLS 1.2.

Answer the following questions, considering the security implications <u>in this specific scenario only</u>:

[1.5 points] 1. Identify and describe the 2 most relevant assets at risk in such a scenario, along with their associated threats and threat agents. All elements must be motivated.

Asset	**Threat**	**Threat Agent**
Central Traffic Control System	***Unauthorized Configuration Manipulation**: The use of FTP, which lacks encryption, exposes the system to the risk of tampered configuration files or unauthorized updates to traffic signals, digital signage, or ramp meters.* ***Denial of Service (DoS) Attacks**: Attackers could flood the control system with malicious traffic, causing it to crash and leaving the city unable to manage traffic effectively.*	***Hackers**: Using tools to intercept and alter FTP communication or launch DoS attacks to disrupt traffic management.* ***Terrorists or Malicious Actors**: Exploiting the insecure system to create chaos, disrupt traffic flow, or cause accidents.*
IoT Sensor Network	***Data Interception and Manipulation**: WEP encryption is highly insecure and susceptible to cracking, allowing attackers to intercept or modify sensor data, such as vehicle counts or weather information.* ***Device Hijacking**: Exploiting weak security to take control of sensors and send falsified data to the central control system.*	***Cybercriminals**: Using widely available tools to crack WEP encryption and gain unauthorized access to sensor communications.* ***Insiders**: Employees or contractors with insider knowledge could manipulate sensor data for malicious purposes.*

[1.5 points] 2. Identify 3 potential vulnerabilities in the company's operations, specifying how they can be exploited by threat agents and a possible countermeasure.

Vulnerability	Exploit	Countermeasure
Insecure Communication Protocols (FTP and WEP)	Attackers can intercept FTP communications to modify traffic management configurations. Similarly, WEP encryption in IoT devices is vulnerable to cracking, allowing attackers to eavesdrop or inject malicious data.	Replace FTP with secure protocols like SFTP or HTTPS for traffic control updates. Upgrade Wi-Fi security from WEP to WPA3 for IoT sensor communication.
Legacy SQL Database on the AWS Server	Attackers could exploit SQL injection vulnerabilities in the legacy database to gain unauthorized access, extract sensitive traffic data, or disrupt the database's operation.	Implement input validation and parameterized queries to prevent SQL injection. Migrate to a more secure and updated database system with regular patches and encryption for stored data.
Outdated TLS Version (TLS 1.2) on the Web Portal	If TLS 1.2 is configured improperly or lacks updated cipher suites, attackers could exploit vulnerabilities in outdated cryptographic protocols to intercept or manipulate communication between users and the web portal. This could lead to session hijacking or data breaches, especially for government staff accessing the system.	Upgrade the web portal to TLS 1.3, which offers improved security features, faster handshakes, and resistance to vulnerabilities in older TLS versions. Regularly monitor and update cryptographic configurations to maintain compliance with modern security standards.

[1 point] 3. As a password recovery scheme, the company implemented the following mechanism: when a user asks to reset their password, an email is sent to the user with the user's password. Analyze the security of this password recovery scheme and identify its potential weaknesses and countermeasures.

Weaknesses of the Current Password Recovery Scheme:

1. **Exposes Passwords via Email**:
    - Sending passwords in plaintext via email risks interception if the email is transmitted over insecure channels or if the user's email account is compromised.
2. **Insecure Password Storage**:
    - The ability to retrieve and send passwords indicates that passwords are likely stored in plaintext or using reversible encryption, which is a significant security flaw.
3. **Encourages Poor Security Practices**:
    - Users may reuse passwords across multiple platforms, increasing exposure if the email

is intercepted or if another account is compromised.

Potential Exploits:

- **Account Takeover**: If an attacker gains access to the user's email account, they can request a password reset and access the account directly using the plaintext password.

Countermeasure:

- Replace the current scheme with a secure password recovery process:
    1. Send a unique, time-limited password reset link to the user's email.
    2. Require the user to create a new password through a secure web interface.
    3. Store passwords securely using modern hashing algorithms like bcrypt or Argon2.

Exercise 2 - Introduction to Cryptography (6 points)

[3 points] Consider the problem of performing periodic firmware updates to an air-gapped machine (i.e., a machine without network connection available to it). The machine is endowed with an USB port, an operating system of your choice, a timezone-synchronous reliable clock. Design a simple cryptographic protocol to validate the firmware updates, considering, as an attacker model, everything transporting the firmware from the source to the machine itself.

Full marks will be awarded to the solution requiring the least computationally expensive approach, and providing resilience against performing a memory dump from a single machine and minimizing key management. You can assume that each machine has a unique serial number available to the OS in a non-tamperable form.

Solution: The problem at hand is to guarantee the integrity of the firmware (confidentiality is not required, although, it won't hurt if you also provide that). Constraints on the computational power favour a solution based on symmetric primitives alone, such as the following one:

- The machine manufacturer keeps a single master secret of sufficient length (e.g. 512 b)
- For each machine, derive a symmetric key through hashing together the unique ID and the master secret (from now on, called m-key)
- Embed the m-key in the machine firmware; if possible encrypt the machine mass memory to hinder cold-dumps
- Each firmware update for the machine is endowed with a MAC tag, computed with the m-key
- Before updating the firmware, the machine checks the tag validity

[3 points] Current best practices from the Certification Authority Browser Forum (CA/Browser Forum) state that CAs should provide certificates with a lifetime not exceeding three years. A recent proposal by Let's Encrypt, a non-profit CA aims at reducing the maximum lifetime of the certificates down to 90 days. State, synthetically, which improvements to the PKI security and efficiency the proposal would bring, and what issues could arise.

Solution: Improvements: i) the exploitation window for a compromised private key is reduced even if revocation mechanisms fail for any reason; ii) certificate revocation lists grow significantly shorter, as they only contain serial numbers for certificates which are still valid.
Issues: i) Certificate lifetime becomes shorter than the one of (essentially any) digital device employing them. Devices with an expected shorter lifetime have now to accommodate a certificate renewal mechanism; ii) The volume of signatures per unit of time to be made by the CA is essentially multiplied by 12; most secure accelerators are expected to be coping with this, but a bump by an order of magnitude is non trivial.

Exercise 3 - Binary Vulnerabilities (6 points)

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <stdbool.h>
4.
5. typedef struct {
6.     int count ;
7.     char c;
8.     char buffer[400];
9.     bool isMichael;
10.  } paperCompany;
11.
12. int main() {
13.     paperCompany company = {
14.         .count = 0,
15.         .c = '\0',
16.         .buffer = {0},
17.         .isMichael = false
18.     };
19.
20.     printf("Welcome to Dunder Mifflin! Type away. Type '!' to exit.\n");
21.
22.     while (company.count <= 400 && (company.c = getc(stdin)) != '!') {
23.         company.buffer[company.count++] = company.c;
24.
25.         // Michael's responses to certain characters
26.         switch (company.c) {
27.             case '\n':
28.                 printf("You've typed: ");
29.                 printf(company.buffer);
30.                 printf("And just as you left your input, I... declare...
bankruptcy!\n");
31.                 break;
32.             case ' ':
33.                 printf("Sometimes I'll start a sentence and I don't even
know where it's going. I just hope I find it along the way.\n");
34.                 break;
35.             case '?':
36.                 printf("You know what they say. Fool me once, strike one,
but fool me twice...strike three.\n");
37.                 break;
38.             case '#':
39.                 printf("I'm not superstitious, but I am a little
stitious.\n");
40.                 break;
41.         }
```

```
42.     }
43.
44.     if (company.isMichael) {
45.         printf("And I knew exactly what to do. But in a much more real
sense, I had no idea what to do.");
46.         system("/bin/sh");
47.     }
48.
49.     return 0;
50. }
51.
```

*The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **cdecl** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against the exploitation of memory corruption errors are present (unless specified otherwise).*
Additional info:

- **getchar**() reads the next character from stream and returns it as an unsigned char cast to an int, or EOF on end of file or error.

[1 point] 1. The program might be affected by typical buffer overflow and format string vulnerabilities. Complete the following table, focusing on a vulnerability per row.

Vulnerability	**Line**	**Motivate and Modify the line to solve the detected vulnerability**
Buffer Overflow	**[0.25]** **22**	**See Slides [0.25]**
Format String	**[0.25]** **29**	**See Slides [0.25]**

[2 points] 2. Write an exploit that **must execute the following shellcode**: \xff\xff\xff\xff\xff\xff\xff\xff.
2.a) Describe **all the steps and assumptions** required to successfully exploit the vulnerability. Include also any assumption on how you must call and run the program: e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables (if any), the input provided during the execution, if multiple executions are necessary.

You can exploit the format string vulnerability by writing the shellcode into the variable buffer and then using the format string to hijack the execution flow to the shellcode inside the buffer.

To correctly execute the line of code vulnerable to the format string vulnerability, you must insert a `\n` *character after the exploit.*

Moreover, the payload must not contain any `!` *in the middle, as this will exit the while loop, causing the program to return prematurely. At the same time, you need to place a* `!` *at the end to allow the program to return to the overwritten saved EIP.*

2.b) Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation showing:
- Direction of growth and high-low addresses;
- The name of each allocated variable;
- Any relevant content;
- The functions stack frames.

Show also the content of the caller frame.
Assume the compiler pads any variable that is smaller than or not a multiple of 4 bytes to align with a 4-byte boundary.

[2 points] 3. Ryan decides to compile the program with the "**NX"** mitigation. Is it effective to prevent your exploit?
If it is not effective, please explain why. Otherwise, if the mitigation technique is effective, explain why and discuss whether it is possible to **spawn a shell** in a different way. Write the full exploit's details, otherwise explain why it cannot be exploited.
Motivate your answer with all the necessary details.

It is effective. See slides…

You can use the one-byte buffer overflow to overwrite the boolean variable `isMichael` *with a value that is not zero. You just need to input 401 values. The last character will overwrite the boolean value.*

You must not insert any `!` *in the middle of the payload; otherwise, the while loop will stop prematurely, preventing the overflow.*

[1 points] 4. Ryan decides to enable the "ASLR" mitigation, and only that one. Is there a way to still **execute the arbitrary shellcode of point 2**?
If not, please explain why. Otherwise, write the full exploit's details.
Motivate your answer with all the necessary details.

If no `!` *characters are inserted in the buffer, the vulnerable line for the format string can be called as many times as we want. We just need to insert more* `\n` *characters at strategic points. This means we can use a format string to leak the stack addresses in one iteration and then write the same exploit at point 2 with the newly leaked address.*

Exercise 4 - Web Vulnerabilities (6 points)

StagePass is a new web platform for purchasing concert tickets. You can charge your account through your credit card and buy the tickets for the concerts you like. Additionally, you can pick your seat or your zone. Don't miss the next concert! Get your ticket with StagePass!
The relevant code of the application is the following:

```
00  @app.route('/buy', methods=['POST'])
01  def buy(request, session):
02     if not is_session_valid(session):
03          return redirect(url_for("/login"))
04     user_id = session["user_id"]
05     if "concert_id" not in request.args.keys():
06          return render_template("concerts.html", error="No concert specified!")
07     concert_id = request.args["concert_id"]
08     if "seat" not in request.args.keys():
09          return render_template("concerts.html", error="No seat specified!")
10     seat = request.args["seat"]
11     credits_q = "SELECT credits FROM USERS WHERE user_id = ?;"
12     credits_r = execute_query_with_statement(credits_q, user_id)
13     concert_q = "SELECT * FROM CONCERT WHERE concert_id = ?;"
14     concert_r = execute_query_with_statement(concert_q, concert_id)
15     if len(concert_r) != 1:
16         return render_template("concerts.html", error="Concert not found!")
17     concert_price = concert_r[0]["price"]
18     credits = int(credits_r[0]["credits"])
19     if (credits < concert_price):
20         return render_template("concerts.html", error="Not enough credits!")
21     ticket_q = "INSERT INTO tickets VALUES (" + user_id + ", '" + seat + "', " + concert_id
           + ");"
22     execute_query(ticket_q)
23     update_q = "UPDATE user SET credit=credit-" + concert_price + " WHERE concert = " +
           concert_id + ";"
24     execute_query(update_q)
25     band = result[0]["band"]
26     return render_template("concerts.html", success="Thanks for your purchase!",
           price=concert_price, band=band)
```

```
50 <html> <!-- concerts.html -->
    ...
61      <h1>{{ success }}</h1>
61      <p>Band: {{ band }}</p>
61      <p>Price: {{ price }}</p>
    ...
70 </html>
```

The relational database schema used by the website is the following:

Table: **users**

user_id (Primary Key) (int) (auto increment)	username (String)	password (String)	email (String)	credit (int)

Table: **concerts**

concert_id (Primary Key) (int) (auto increment)	price (int)	date (date)	band (string)

Table: **tickets**

user_id (Primary Key) (int) (Reference: users.user_id)	seat (string)	concert_id (Primary Key) (int) (Reference: concerts.concert_id)

Assume the following:

- The session is correctly handled (i.e., you cannot forge arbitrary cookies).
- The function **is_session_valid** correctly checks if the user is logged in.
- The function **db_execute_query_with_statement** executes queries with prepared statements.
- The function **db_execute_query** executes queries without prepared statements.
- The function **render_template** **does not perform any sanitization of the input.**
- The **DBMS** does not support stacked queries (e.g., "SELECT …; INSERT INTO …;" fails if executed)
- The **DBMS** is case-sensitive; "Law" and "law" are two different users.

More details:

- The function **render_template** generates an HTML page from a .html file with optional parameters that are placed in the HTML code.
- The function **url_for** generates a URL of the provided page of the same domain. Optional query parameters of the URL can be specified as parameters.
- The function **redirect** redirects the user to the web page of the provided url.
- The functions **db_execute_query_with_statement** and **db_execute_query** return a list of rows as a result, each of which has the fields specified by the **SELECT** statement. For instance, result[4]["comment"] is accessing the field "comment" of the 5th row.

1. [3 points] Fill out the following table by answering the questions for each class of vulnerabilities. While answering the questions, be **exhaustive, concise, and straight to the point**. If you believe there might be a vulnerability, specify all the corresponding line/s of code.

Vulnerability class	*Is there a vulnerability of this class in the code above? How could an adversary exploit it?*	*If the vulnerability is present, explain the simplest procedure to remove it.*

SQL Injection (SQL-I)	*Line 21. An attacker can inject data into the DB.*	*Using prepared statements.*
cross-site scripting (XSS) **Specify if reflected, stored, DOM...**		
Cross-site request forgery (CSRF)	*CSRF token is not implemented for the function "buy". A user can buy a ticket for others.*	*Implement a CSRF token correctly randomized for the function "buy"*

[1.5 points] 2. One of your friends asks you to open a link. Once opened, you notice that you have a new ticket for a band that you don't like. Can you reproduce the exploit? Explain all the steps you would take to reproduce the exploit, including the final code you would write.

CRSF

....

[1.5 points] 3. You found an exploit that allows you to purchase many tickets, for different concerts and users, by just buying one ticket. Can you show me the exploit for purchasing two tickets at the price of one? One for you, and one for the user *Law*

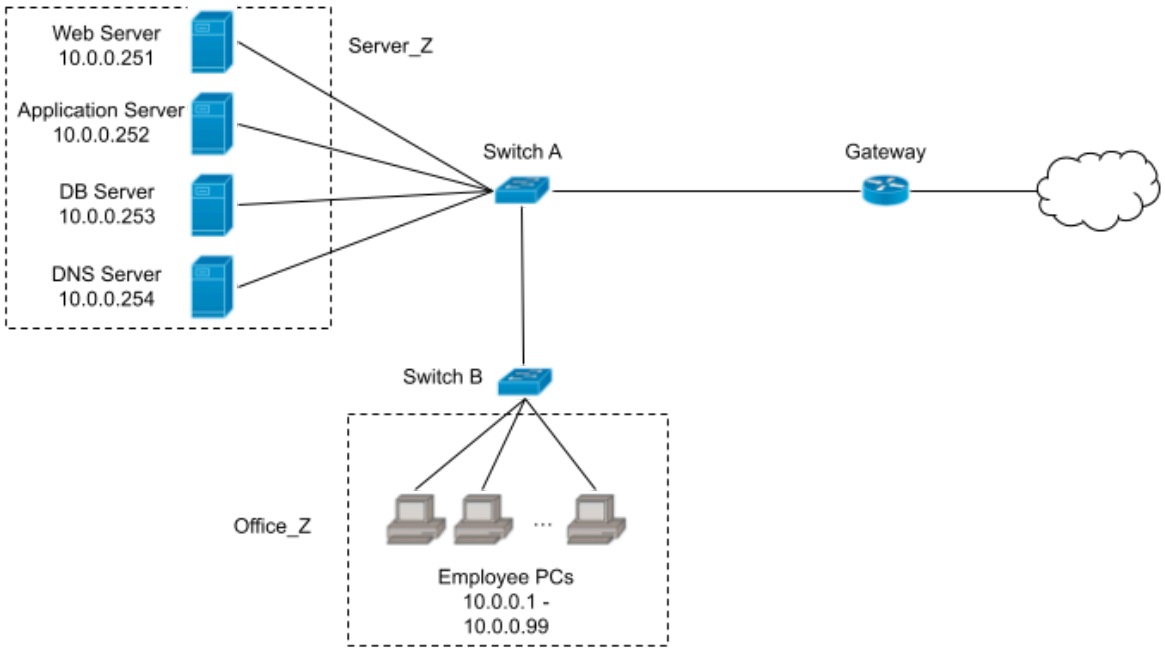
SQLI con subquery

....

Exercise 5 - Network Security (6 points)

GlobeCorp is a leader in freight transport and logistics and has several branches worldwide. Their main branch hosts both administrative offices and their data center. The employees can access an internal custom web service that allows them to view and modify the information about customers and orders. A web server, accessible only from the local network, provides this service, together with an application server and a DB server. A local DNS server is used to resolve the IP addresses for the websites that the employees want to access.

The network layout is schematized in the following figure.



You are the system administrator, and while going through the logs of switch A for a routine check, you notice a suspicious sequence of packets.

Timestamp	Src IP	Dst IP	Description
+00,000s	*10.0.0.2*	*DNS_IP*	*Recursive DNS query for "www.reddit.com"*
+00,001s	*DNS_IP*	*AUTH_DNS_IP*	*Recursive DNS query for "www.reddit.com"*
+00,002s	*140.234.43.234*	*DNS_IP*	*DNS Response: "www.reddit.com" is 10.0.0.12*
+00,003s	*140.234.43.234*	*DNS_IP*	*DNS Response: "www.reddit.com" is 10.0.0.12*
...			
+00,103s	*140.234.43.234*	*DNS_IP*	*DNS Response: "www.reddit.com" is 10.0.0.12*
+00,201s	*DNS_IP*	*10.0.0.2*	*DNS Response: "www.reddit.com" is 10.0.0.12*
+00,312s	*10.0.0.2*	*10.0.0.12*	*HTTP request for "www.reddit.com"*
+10,010s	*AUTH_DNS_IP*	*DNS_IP*	*DNS Response: "www.reddit.com" is 156.212.89.32*

[1 point] 1. Can you identify the type of the ongoing attack? State the name of the attack and explain how it works.

DNS cache poisoning…

[1 point] 2. Who is the attacker? Can their IP be spoofed? Who is the victim?

One of the employees or someone that controls their PC. The IP is spoofed in the requests, but they must provide one real IP, otherwise the attack wouldn't make sense (10.0.0.12 in this situation). The victim is 10.0.0.2.

[2 points] 3. You want to prevent this accident from happening again. Your boss gives you an unlimited budget to buy as many firewalls as you wish and complete freedom on where to place them in the network. How many firewalls would you buy, and where would you place them?

One is sufficient, and it should replace switch A.

Write the firewall rules you deem necessary, making them consistent with your choice and assuming you are working with a stateful packet filter.

Firewall	Src IP	SRC Port	Direction	Dst IP	Dst PORT	Policy	Description
FW1	*all*	*any*	*all*	*all*	*all*	*Deny*	*Default Deny*
FW1	*User_IP*	*any*	*Office_Z -> Server_Z*	*DNS_IP*	*53*	*Allow*	*Allow DNS requests*
FW1	*Office_IP*	*any*	*Office_Z-> Server_Z*	*WS_IP*	*80/443*	*Allow*	*Allow access to WS*
FW1	*Office_IP*	*any*	*Office_Z -> Internet*	*any*	*80/443*	*Allow*	*Enable internet connection for employees*

[1 point] 4. The employees from other branches need to access the internal custom web service as well. As the system administrator, what solution would you adopt to make it possible? Motivate your answer.

Use a VPN...

Exercise 6 - Malware (6 points)

Akihiko Kayaba, the inventor of the Sword Art Online video game, contacts you during its development. A competitor company has infected the beta servers with malware. Included here is a piece of the malware's code. Kayaba is not interested in understanding what the malware does but rather in how to prevent future infections and how to analyze the sample effectively.

```
char *buffer;
char *key = "KEYKEYKEY_KEYKEY"
size_t len = 0x1a0;

void sig_handler(int signumber) {
      char * decrypted = decrypt(buffer, key, bytecode_len);
      decrypted();
}

int main() {
      // Sets an handler for floating point exceptions
      signal(SIGFPE, sig_handler);
      float result = 1 / 0;

      return 0;
}
```

Some information useful in analyzing the piece of code:

- The function *signal(SIGNUMBER, handler)* registers a handler that is executed when the operating system sends the signal *SIGNUMBER* signal to the program.
- A division by zero triggers the operating system to raise a *SIGFPE* signal.
- By default, debuggers pass only non-error signals, such as *SIGALRM*, to the debugged program. Error signals, like *SIGFPE*, are intercepted and not passed to the program being debugged.
- Call to *decrypted()* starts the malicious actions of the malware.

1. [2 Points] Describe the **two** anti-analysis techniques employed by the malware, specifying their category, what each technique does, the type of analysis it targets, and how it complicates that analysis. Incomplete or vague answers will receive no points, and incorrect answers will result in point deductions.

Anti-debugging and polymorphism: The division by zero triggers a SIGFPE signal, which is caught by the handler. This handler decrypts the payload and initiates the malware's malicious actions.
Anti-debugging complicates dynamic analysis because the signal must be passed to the program, which debuggers typically intercept. Polymorphism hinders static analysis [see slides for further details].

2. [2 Points] Kayaba's competitors made a critical mistake in the malware distribution: they distributed the same copy over the internet. As a result, analysts were able to register its signature, and it is now detected by all static analysis tools. Kayaba's competitors have contacted you to address this issue. Please suggest how to improve the malware distribution by proposing two different, and correctly implemented, metamorphic and polymorphic versions. Obviously, your new solution must maintain the anti-analysis techniques found at Point 1.

Metamorphic Solution: Generate different metamorphic versions of the malware, all encrypted with the same key. This ensures that the resulting encrypted payload varies across samples, while the malware's actions remain consistent.

Correct Polymorphic Solution: Use a unique encryption key for each distributed sample.

3. [2 Points] Consider the anti-analysis techniques identified in Point 1 and the improvements to malware distribution implemented in Point 2. Describe how you would approach the analysis of this malware. Specifically, detail how you would configure your analysis tools, the type of analysis you would perform (e.g., static or dynamic), and the specific elements you would focus on examining.

Setup dynamic analysis to pass the signal to the program and analyze its behavior in a sandbox. (other solutions are accepted as well if they make sense).